TrailSense Multilateration Library: C API Reference

July 12, 2026

CONTENTS

TrailSense Multilateration Library: C API Reference

The C application programming interface: functions, structs, enums, status codes, and the memory and capacity contract. Part of the TrailSense Multilateration Library documentation set. The position output contract is in the Output Reference.

C API Reference

The public C ABI of `libtrailsense_triangulate`: functions, structs, enums, status codes, memory/thread-safety/capacity contracts. Full surface in `include/trailsense_triangulate.h` (authoritative).

Version macros

Macro	Value	Meaning
`TS_TRI_VERSION_MAJOR`	`1`	Major ABI version.
`TS_TRI_VERSION_MINOR`	`0`	Minor ABI version.

Capacity macros

Macro	Value	Meaning
`TS_MAX_GROUPS`	`64`	Max distinct `(mac_hash, band)` groups per call.
`TS_MAX_NODES_PER_GROUP`	`32`	Max distinct nodes within one group.

Hard per-call limits. Input beyond either is silently dropped (see “Capacity bounds and silent truncation”).

Functions

TS_TRIANGULATE

```
int ts_triangulate(const ts_obs_t *obs, size_t n_obs,
                   const ts_anchor_t *anchors, size_t n_anchors,
                   ts_position_t *out, size_t out_cap, size_t *out_n);
```

Correlates `obs` by `(mac_hash, band)`, keeps the latest sample per node per group, solves each group by its count of distinct enrolled, resolvable anchors: 0 or 1 = no position; exactly 2 = approximate bilateration (confidence 40); 3 or more = robust IRLS multilateration (confidence 70

for exactly 3, 85 for 4 or more). Anchor count at solve time is the only selector, so a 3rd resolvable unit auto-upgrades bilateration to full triangulation. Writes up to `out_cap` positions into `out`, sets `*out_n` to the count.

Parameter	Type	Direction	Meaning
`obs`	`const ts_obs_t *`	in	Raw observations. Read-only, not retained.
`n_obs`	`size_t`	in	Elements in `obs` (reads exactly this many).
`anchors`	`const ts_anchor_t *`	in	Surveyed anchors. Read-only, not retained.
`n_anchors`	`size_t`	in	Elements in `anchors`.
`out`	`ts_position_t *`	out	Caller-owned buffer. On success `out[0 .. *out_n - 1]` valid.
`out_cap`	`size_t`	in	Capacity of `out`, elements.
`out_n`	`size_t *`	out	Positions written.

Returns `TS_OK` (0) on success (including zero positions) or a negative `ts_status_t`. On any negative return, do not read `out` or `*out_n`.

TS_BAND_PRIOR

```
int ts_band_prior(uint8_t band, int32_t *tx_dbm, int32_t *a_cdb, int32_t
*n_milli);
```

Returns the OUTDOOR band-prior defaults for a band, to inspect or seed. Each output pointer may be NULL to skip.

Parameter	Type	Direction	Meaning
`band`	`uint8_t`	in	Band 0/1/2 (see `ts_band_t`).
`tx_dbm`	`int32_t *`	out, optional	Assumed transmit power, dBm. NULL to skip.
`a_cdb`	`int32_t *`	out, optional	Path-loss reference `A`, centi-dB (hundredths of a dB). NULL to skip.
`n_milli`	`int32_t *`	out, optional	Path-loss exponent `n`, milli (thousandths). NULL to skip.

Returns `TS_OK` (0), or `TS_ERR_BAD_BAND` (-3) if `band` is outside `0..2`.

OUTDOOR defaults returned:

Band	`tx_dbm`	`a_cdb` (centi-dB)	`n_milli` (milli)
0 WIFI	18	4000 (40.00 dB)	2000 (2.000)
1 BLE	8	4000 (40.00 dB)	2000 (2.000)
2 CELL	23	3800 (38.00 dB)	2000 (2.000)

Enums

TS_BAND_T

```
typedef enum {
    TS_BAND_WIFI = 0,
    TS_BAND_BLE  = 1,
    TS_BAND_CELL = 2
} ts_band_t;
```

Radio band. Values match the low 2 bits of the wire `band_chan` field. The `band` fields on `ts_obs_t` and `ts_position_t` carry one as a `uint8_t`. Any band outside `0..2` is rejected: `ts_triangulate` returns `TS_ERR_BAD_BAND`.

TS_STATUS_T

```
typedef enum {
    TS_OK                = 0,
    TS_ERR_NULL_ARG      = -1,
    TS_ERR_OUT_TOO_SMALL = -2,
    TS_ERR_BAD_BAND      = -3
} ts_status_t;
```

Code	Value	Meaning
`TS_OK`	`0`	Success. `*out_n` positions written (may be 0).
`TS_ERR_NULL_ARG`	`-1`	A required pointer argument was NULL.
`TS_ERR_OUT_TOO_SMALL`	`-2`	`out_cap` too small for positions produced. No usable partial result.
`TS_ERR_BAD_BAND`	`-3`	An observation carried a band outside `0..2`.

Codes are negative; success is 0. Always branch on `rc != TS_OK` before reading any output.

Structs

TS_OBS_T

One raw observation (one node's sighting of one device).

```
typedef struct {
    int32_t  node_id;
    uint8_t  mac_hash[4];
    uint8_t  band;
    int8_t   rssi;
    uint32_t ts_ms;
} ts_obs_t;
```

Field	C type	Unit	Meaning	Required
`node_id`	`int32_t`	none	Observing node id; matched against `ts_anchor_t.node_id`.	required
`mac_hash`	`uint8_t[4]`	none	4-byte device fingerprint hash; groups by device.	required
`band`	`uint8_t`	none	Radio band (`ts_band_t`, 0/1/2); part of grouping key.	required
`rssi`	`int8_t`	dBm	Signal strength of this sighting.	required
`ts_ms`	`uint32_t`	ms	Timestamp; used only for latest-per-node selection and age-weighting within a group, never to subdivide the batch into sub-windows (caller owns windowing).	required

All five carry meaning; none optional.

TS_ANCHOR_T

One surveyed anchor (sensor node) position, plus optional per-anchor path-loss override.

```
typedef struct {
    int32_t  node_id;
    double   lat;
    double   lon;
    int32_t  path_loss_a_cdb;
    int32_t  path_loss_n_milli;
    uint8_t  enrolled;
} ts_anchor_t;
```

Field	C type	Unit	Meaning	Required / default
`node_id`	`int32_t`	none	Node id; matched against `ts_obs_t.node_id`. If duplicated, last entry wins.	required
`lat`	`double`	degrees	Surveyed latitude (WGS 84).	required
`lon`	`double`	degrees	Surveyed longitude (WGS 84).	required
`path_loss_a_cdb`	`int32_t`	centi-dB	Per-anchor path-loss reference `A`; `<= 0` means use band-prior default.	optional, default via band prior
`path_loss_n_milli`	`int32_t`	milli	Per-anchor path-loss exponent `n`; `<= 0` means use band-prior default.	optional, default via band prior
`enrolled`	`uint8_t`	none	`0` excludes this anchor entirely; any nonzero includes it.	required (set to 1 to include)

The `<= 0` sentinel is a caller-omission convenience. Transmit power always comes from the band prior; only `A` and `n` accept per-anchor overrides. Override values are positive-only in practice.

TS_POSITION_T

One triangulated position.

```
typedef struct {
    uint8_t  mac_hash[4];
    uint8_t  band;
    double   lat;
    double   lon;
    double   accuracy_m;
    int32_t  measurement_count;
    int32_t  confidence;
    int8_t   peak_rssi;
    double   error_semi_major_m;
    double   error_semi_minor_m;
    double   error_major_axis_deg;
} ts_position_t;
```

Field	C type	Unit	Meaning
`mac_hash`	`uint8_t[4]`	none	Device fingerprint hash (from group key).
`band`	`uint8_t`	none	Band (`ts_band_t`).
`lat`	`double`	degrees	Estimated latitude. Emitted before `lon` . WGS 84.
`lon`	`double`	degrees	Estimated longitude.
`accuracy_m`	`double`	meters	Weighted CEP (circular error probable). 2-node fix equals `error_semi_major_m` .
`measurement_count`	`int32_t`	count	Participating anchors (distinct enrolled, resolvable anchors that saw the device). NOT the post-outlier survivor count.
`confidence`	`int32_t`	none	Geometry tier, not a probability: `40` (exactly 2, approximate bilateration), `70` (exactly 3), `85` (4 or more).
`peak_rssi`	`int8_t`	dBm	Strongest participant RSSI.
`error_semi_major_m`	`double`	meters	Error-ellipse semi-major axis. See below.
`error_semi_minor_m`	`double`	meters	Error-ellipse semi-minor axis. See below.
`error_major_axis_deg`	`double`	degrees	Bearing of semi-major axis: 0 = North, clockwise, folded into `[0, 180)` .

Ellipse semantics differ by anchor count and are the most important consumer detail. 2-node fixes are MIXED (cross axis is a hard bound, NOT a confidence radius); 3-or-more fixes are an isotropic placeholder. See the Output Reference for full ellipse semantics.

Memory-ownership contract

Caller owns every buffer. The library allocates nothing on the heap and keeps no global state.

- `obs` and `anchors` are borrowed read-only for the call; nothing retained after return.

- `out` is written by the library; caller allocates it (stack, `.bss`, or heap) and passes capacity as `out_cap`.
- After return, the library holds no reference to any caller buffer; free or reuse all immediately.

Thread-safety

Reentrant. No internal locks, no shared mutable state, so distinct threads may call `ts_triangulate` concurrently provided each uses its own `obs`, `anchors`, and `out` buffers. Do not share a single output buffer across threads during an in-flight call. No cross-call state to protect.

Capacity bounds and silent truncation

At most `TS_MAX_GROUPS` (64) distinct `(mac_hash, band)` groups per call, and `TS_MAX_NODES_PER_GROUP` (32) distinct nodes within one group. Input beyond either is silently dropped: groups past the 64th, and nodes past the 32nd within a group, are discarded, yielding a partial result that still returns `TS_OK`. The core never prints and never signals truncation.

A direct library linker gets no warning and must keep each batch under both caps. If a batch could exceed 64 groups, or 32 nodes for one device, split it and call once per batch. (The bundled CLI adds its own stderr warning for over-cap input and still exits 0 with the partial result; the library does not warn.)

Sizing the output buffer and TS_ERR_OUT_TOO_SMALL

At most one position per distinct `(mac_hash, band)` group, and at most `TS_MAX_GROUPS` groups per call, so an `out` buffer of `TS_MAX_GROUPS` (64) elements always holds every position one call can emit.

If positions exceed `out_cap`, `ts_triangulate` returns `TS_ERR_OUT_TOO_SMALL` with no usable partial result: do not read `out`. A smaller buffer is safe only when you know the batch has fewer resolvable groups. When in doubt, size `out` to `TS_MAX_GROUPS`.